PROYECTO INTERMODULAR

Diseño y administración de una base de datos para un Contact Center

Módulo 0372 - Gestión de Bases de Datos

Autor: **Sergio A. Tellez Tellez**
Grado Medio Administración de Sistemas Informáticos en Red
Proyecto: *Arquitectura de un Contact Center*

Abril 2026

Índice

1. Introducción
2. Objetivos
3. Descripción del entorno
4. Arquitectura de la red
5. Fase 1 — Análisis del negocio
6. Fase 2 — Diseño del modelo E/R
7. Fase 3 — Creación de la base de datos
8. Fase 4 — Tablas principales
9. Fase 5 — Relación cliente / campaña
10. Fase 6 — Tabla núcleo: llamada
11. Fase 7 — Normalización y FK
12. Fase 8 — Inserción y depuración
13. Fase 9 — Consultas y KPIs
14. Fase 10 — Backup, usuarios y permisos
15. Pruebas de funcionamiento
16. Validación de resultados
17. Problemas y soluciones
18. Buenas prácticas aplicadas
19. Conclusiones

Introducción

¿De qué trata?

Este proyecto pone en marcha la base de datos de un **Contact Center**: clientes, agentes, campañas y, sobre todo, las llamadas, que son el corazón del negocio.

¿Por qué elegí este tema?

Porque profesionalmente me he movido en este tipo de entornos. He visto cómo funciona un call center por dentro (INBOUND, OUTBOUND, turnos, supervisores, KPIs…) y eso me da una ventaja clara: entiendo el porqué de cada decisión técnica y no solo el cómo. Trabajar con datos que ya conozco me permite enfocarme en construir un modelo coherente y útil.

Contact Center

Sector profesional elegido

MySQL Workbench

Entorno de trabajo

Modelo relacional

Enfoque del diseño

Datos reales

Inserciones de negocio

Objetivos

Qué quiero conseguir

Diseñar, construir y administrar la base de datos de un Contact Center, demostrando que puedo pasar de una idea de negocio a un modelo relacional funcional y mantenible.

Modelar el negocio

Identificar entidades reales de un call center y traducirlas a tablas.

Construir en SQL

Crear base de datos, tablas, claves primarias, foráneas e índices.

Gestionar los datos

Insertar, depurar y mantener datos coherentes de INBOUND y OUTBOUND.

Explotar la información

Consultas con JOIN, GROUP BY y CASE para medir la actividad y la efectividad.

Administrar como un profesional

Copias de seguridad, usuarios y permisos por rol.

Aplicar buenas prácticas

Nomenclatura snake_case, FK con nombres propios y mínimo privilegio.

Descripción del entorno

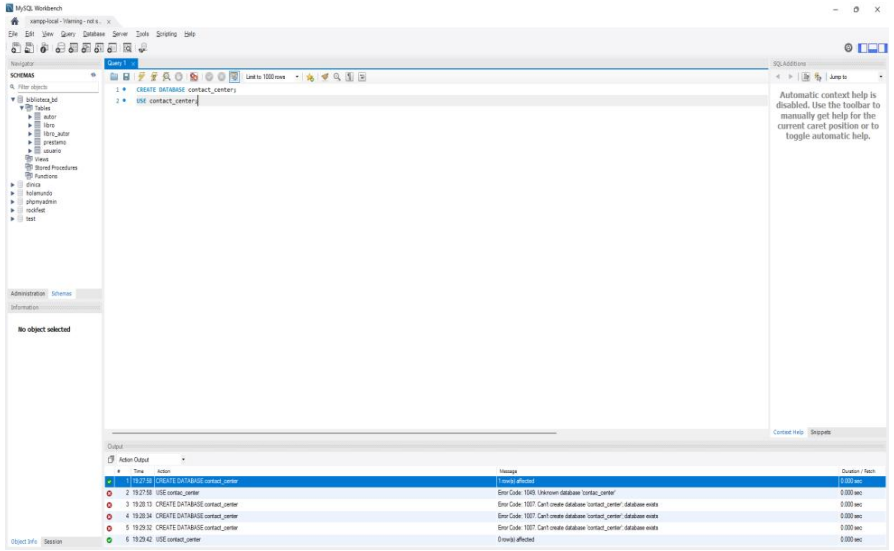
Contexto de negocio

Un Contact Center gestiona la relación con el cliente en dos grandes tipos de operación:

- **OUTBOUND:** llamadas salientes para vender (tarjetas de crédito, créditos de consumo).
- **INBOUND:** llamadas entrantes de atención al cliente (por ejemplo, soporte a pedidos de logística).

Entorno técnico

Todo el trabajo se ha realizado sobre **MySQL 8** usando **MySQL Workbench** sobre un servidor XAMPP local. El esquema se llama contact_center y se documenta con un script .sql que sirve tanto de copia como de instalador.



Entorno MySQL Workbench sobre XAMPP local.

Coherencia con la red

Los agentes son usuarios del dominio y los departamentos se alinean con las VLANs del proyecto de Redes. El modelo de datos encaja con la capa de red y sistemas.

Arquitectura de la red

Agentes

Equipos cliente
VLAN operación

Supervisor

Estaciones de control
VLAN supervisión

Admin DB

Acceso con control total
conexión local

Red corporativa

Servidor MySQL

Esquema contact_center

Tablas: cliente · agente · campana · cliente_campana · llamada

Modelo cliente-servidor

Los puestos de trabajo (agentes y supervisor) consultan el servidor de base de datos a través de la red interna. El administrador trabaja sobre localhost para las tareas de mantenimiento.

Separación por roles

El acceso no es libre: cada perfil se conecta con un usuario distinto del gestor (agente_user, supervisor_user, admin_cc) y cada uno solo puede hacer lo que su rol permite.

Encaje con el proyecto de Redes

El esquema respeta las VLANs ya definidas: el campo departamento de la tabla agente se corresponde con los departamentos de red del entorno.

Análisis del negocio

Entender antes de modelar

FASE

1

Qué hicimos

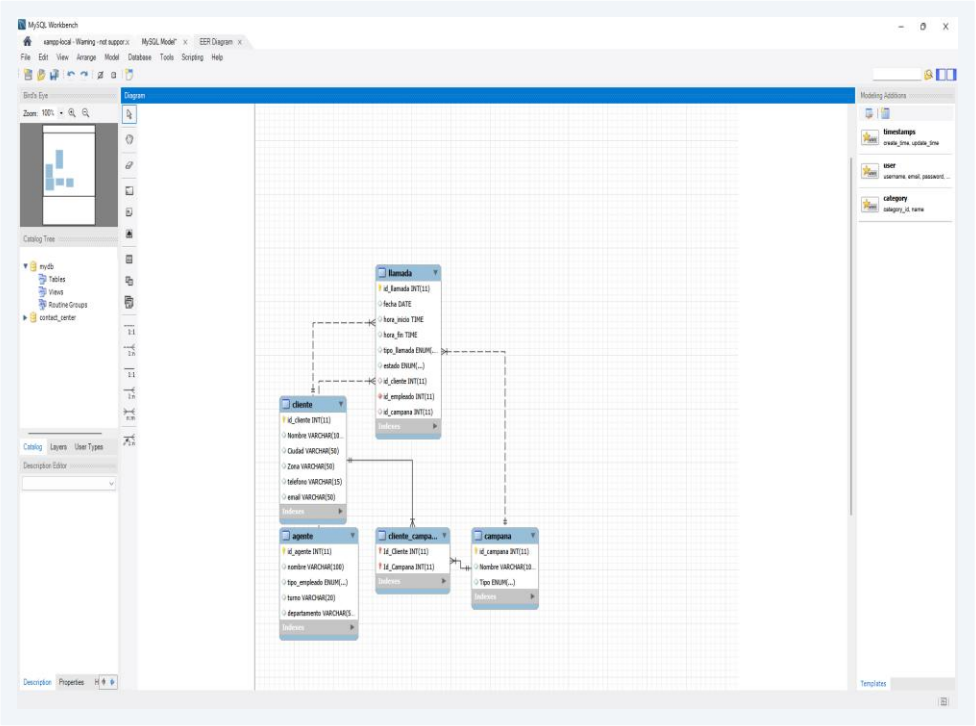
Identificamos los actores y procesos de un Contact Center: clientes, agentes, supervisores, campañas y llamadas (INBOUND y OUTBOUND).

Por qué

Si no entiendes cómo trabaja el negocio, el modelo no sirve. El objetivo aquí no era técnico, era dejar claro qué información merece la pena guardar.

Puntos clave

- Actores: clientes y empleados (agentes / supervisores).
- Procesos clave: gestión de llamadas, campañas y resultados.
- Fuera de alcance: nómina, asistencia o RRHH.
- Coherencia con la infraestructura (dominio, VLANs, departamentos).



Visión final del modelo que se iba a construir.

Diseño del modelo E/R

De la idea al diagrama

FASE
2

Qué hicimos

Se plantea el modelo entidad-relación con cuatro entidades principales (cliente, agente, campana, llamada) y una tabla intermedia para relaciones N:M.

Por qué

Dibujar antes de escribir SQL evita decisiones apresuradas. La tabla llamada queda como eje central del sistema: todo gira en torno a ella.

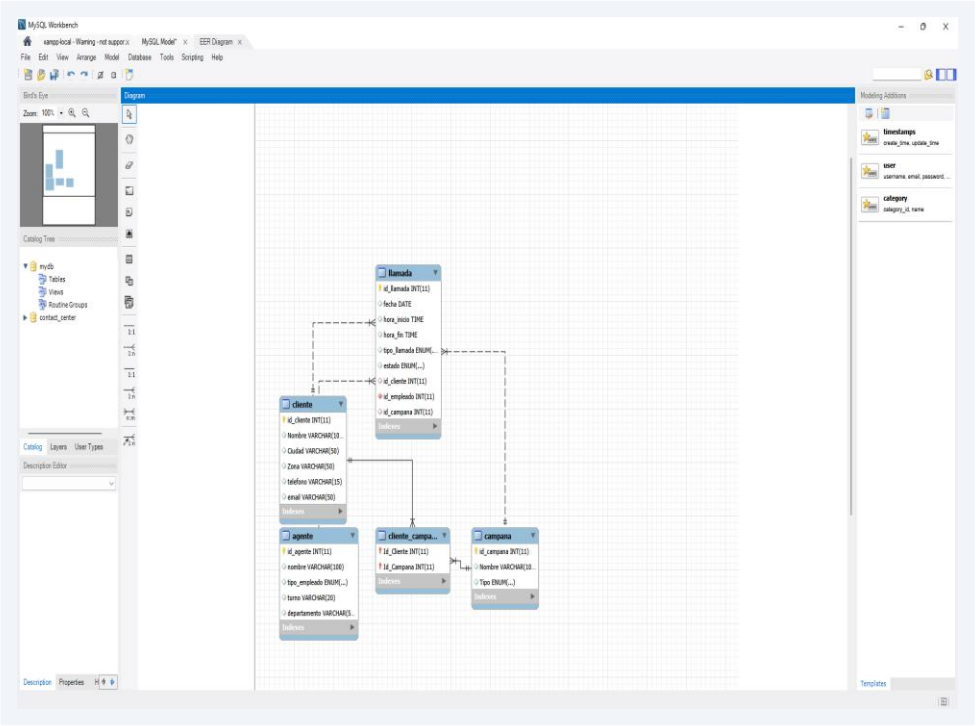


Diagrama E/R del modelo contact_center.

Puntos clave

- Entidad central: llamada — cada interacción real.
- Relaciones 1:N entre llamada y cliente, agente y campana.
- Relación N:M entre cliente y campana (una tabla intermedia).
- Claves primarias autoincrementales para cada entidad.

Creación de la base de datos

CREATE DATABASE contact_center

FASE

3

Qué hicimos

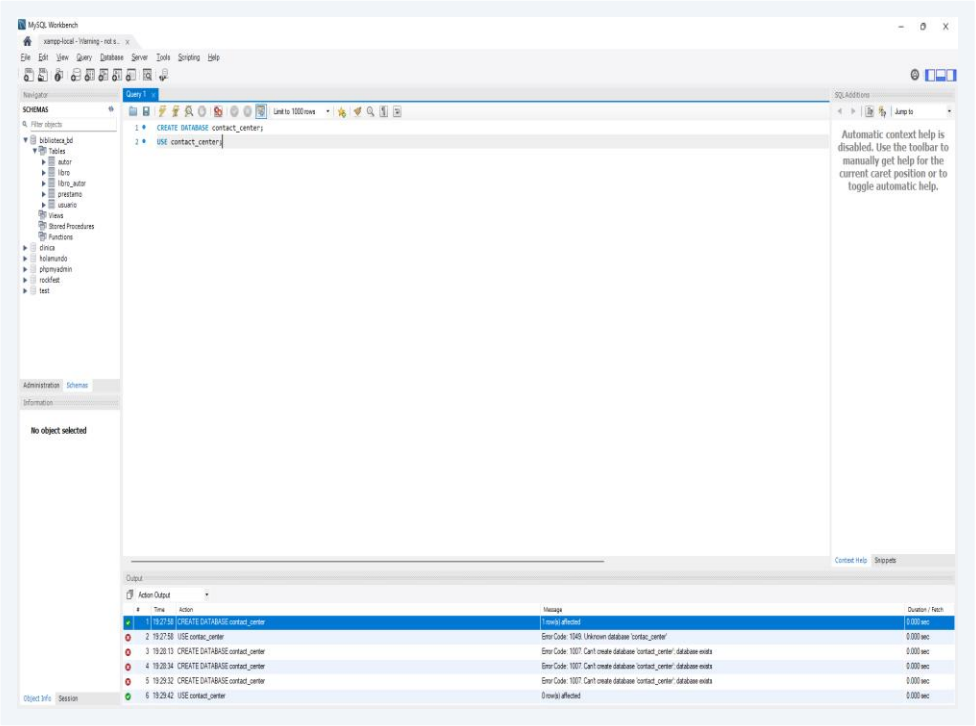
Se crea el esquema contact_center con CREATE DATABASE y se selecciona con USE para trabajar siempre dentro del mismo contexto.

Por qué

Separar la información por esquemas permite tener varios proyectos en el mismo servidor sin mezclarlos. Es el primer paso de cualquier entorno serio.

Puntos clave

- CREATE DATABASE contact_center;
- USE contact_center; para fijar el contexto.
- Refrescar el árbol de schemas en Workbench para verla.
- Aparecieron errores de tipeo (contac_center) — corregidos y documentados.



Creación del esquema en MySQL Workbench.

Tablas principales

cliente, agente, campana

FASE

4

Qué hicimos

Se crean las tres entidades base del sistema con sus atributos mínimos: identificador, nombre y los campos propios de cada una (zona, turno, tipo de campaña...).

Por qué

Antes de relacionar nada hay que tener las entidades bien definidas. Usamos ENUM para los estados cerrados (AGENTE/SUPERVISOR, INBOUND/OUTBOUND) porque son valores fijos del negocio.

Puntos clave

- Claves primarias AUTO_INCREMENT en las tres tablas.
- ENUM en tipo_empleado y tipo de campaña para evitar datos sucios.
- Campos alineados con la realidad del call center (turno, departamento).
- Se renombró 'campaña' a 'campana' para evitar problemas con caracteres.

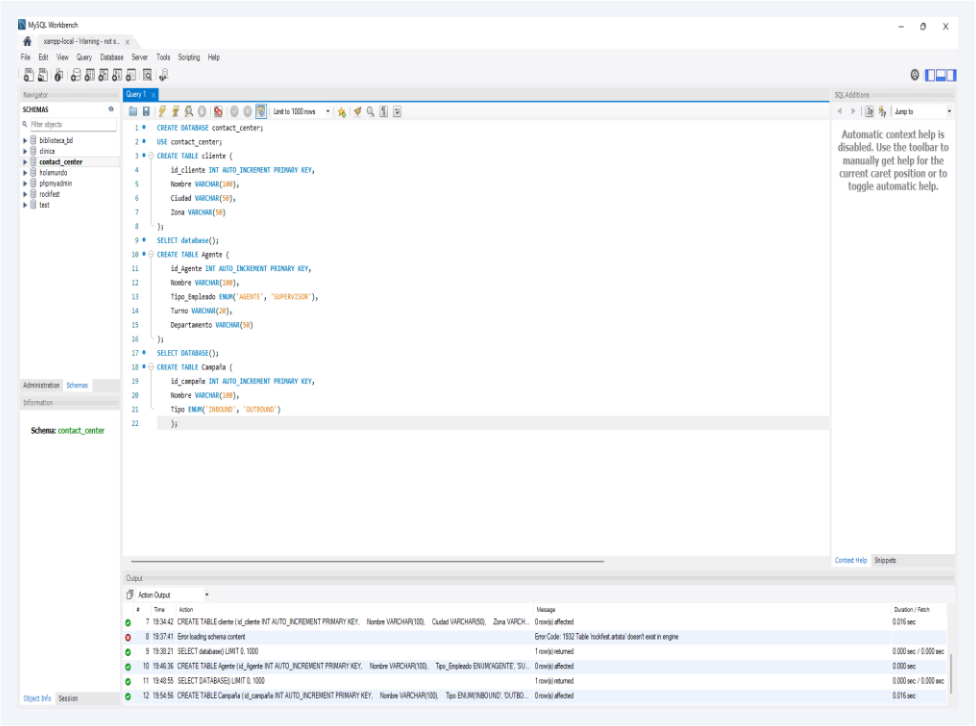
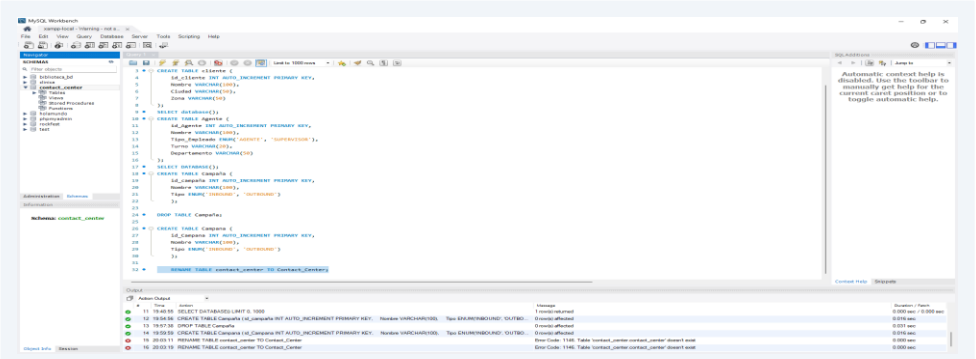


Tabla campana y renombrado para estandarizar.



Relación cliente / campaña

Tabla intermedia N:M

FASE
5

Qué hicimos

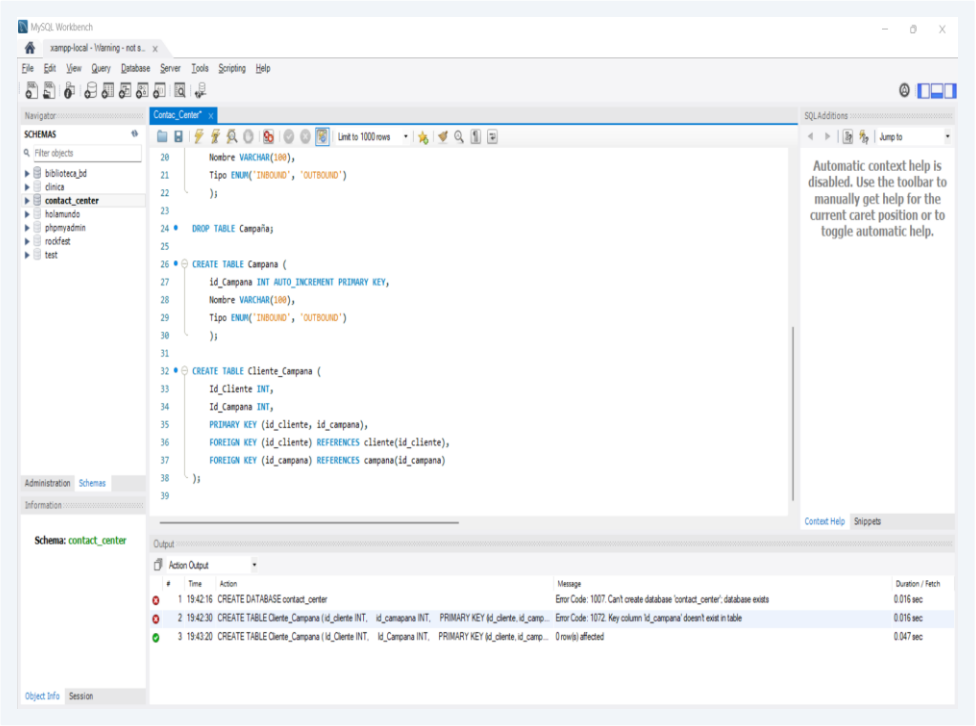
Se crea la tabla intermedia cliente_campana para que un mismo cliente pueda estar en varias campañas y una campaña tener muchos clientes.

Por qué

Un cliente de un contact center casi nunca es exclusivo de una sola campaña. Modelarlo con tabla intermedia es la forma correcta y evita duplicar datos.

Puntos clave

- Clave primaria compuesta (id_cliente, id_campana).
- Dos claves foráneas que apuntan a las tablas originales.
- Integridad referencial: no se puede inscribir a un cliente inexistente.
- Preparado para crecer sin romper el modelo.



Script de creación de la tabla cliente_campana.

Tabla núcleo: llamada

FASE
6

Qué hicimos

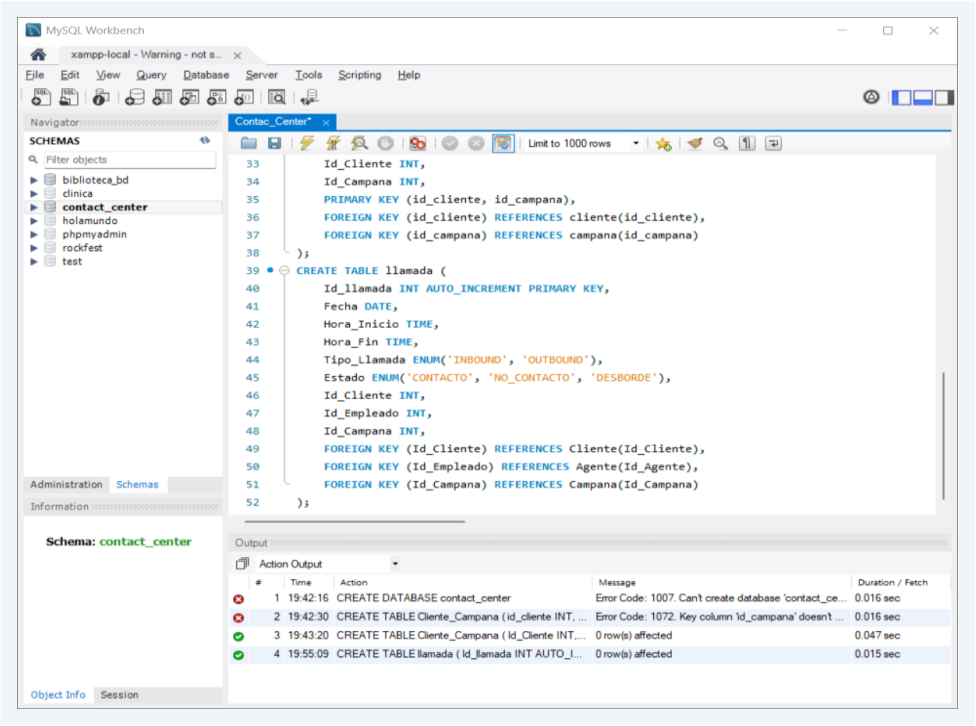
Se crea la tabla llamada con fecha, hora de inicio y fin, tipo (INBOUND / OUTBOUND), estado (CONTACTO / NO CONTACTO) y tres claves foráneas: cliente, agente y campana.

Por qué

Toda la operación del contact center se mide sobre llamadas. Tener esta tabla bien diseñada es lo que después permite sacar KPIs reales.

Puntos clave

- Una fila = una interacción real entre agente y cliente.
- Claves foráneas hacia cliente, agente y campana.
- Tipos de datos DATE y TIME para trazabilidad temporal.
- ENUM para tipo y estado: lista cerrada de valores válidos.



Definición de la tabla llamada con sus FK.

Normalización y FK profesionales

snake_case + nombres propios

FASE

7

Qué hicimos

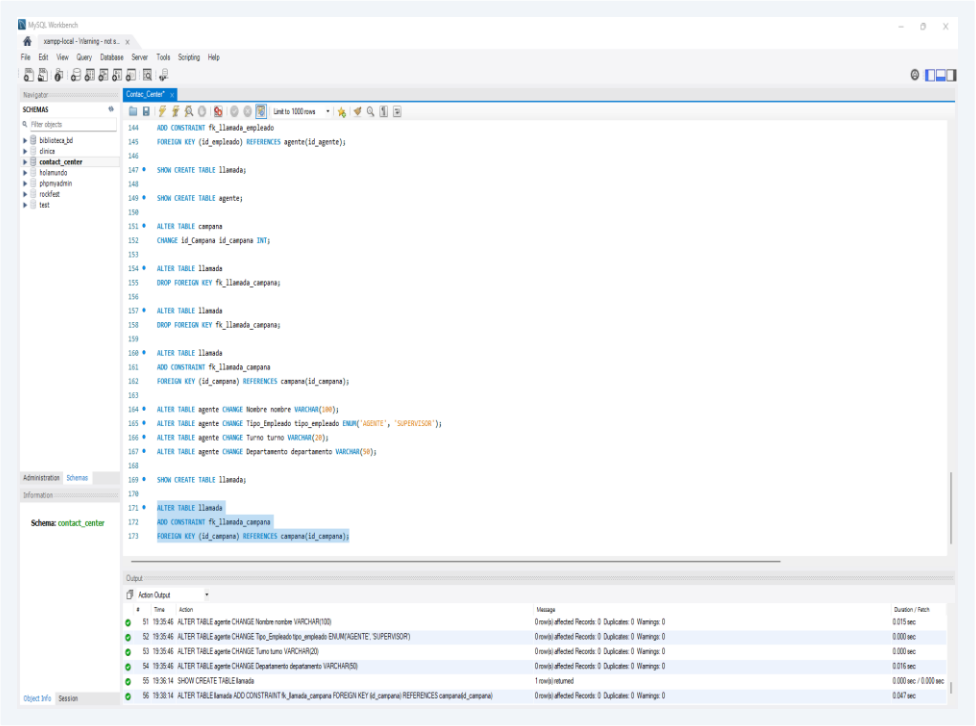
Se estandariza toda la nomenclatura a snake_case y se sustituyen los nombres automáticos de MySQL (llamada_ibfk_1) por nombres propios (fk_llamada_cliente, fk_llamada_empleado, fk_llamada_campana).

Por qué

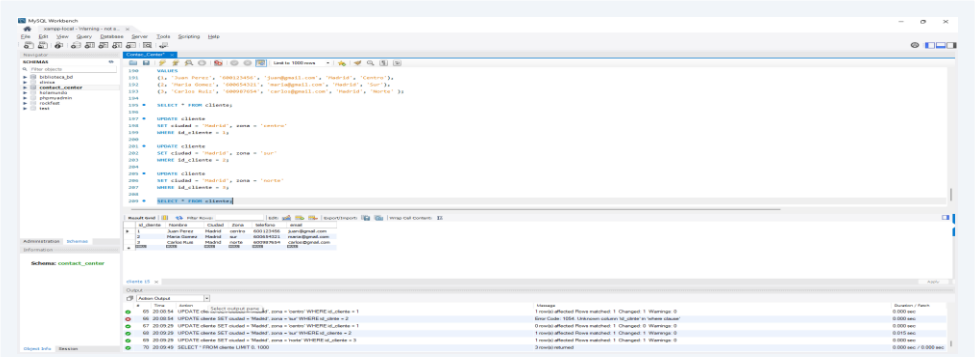
En un entorno real los nombres importan. Un fk_llamada_cliente explica por sí solo qué hace; un llamada_ibfk_1 no dice nada. Además facilita el mantenimiento y la lectura del esquema.

Puntos clave

- Todo en minúsculas, separado con guion bajo.
- FK renombradas con patrón fk_<tabla>_<referencia>.
- Índices creados manualmente donde hacía falta (idx_llamada_*).
- Arreglos sobre claves mal nombradas (id_Campana vs id_campana).



ALTER TABLE aplicando el estándar.



Inserción y depuración de datos

Datos reales, errores reales

FASE

8

Qué hicimos

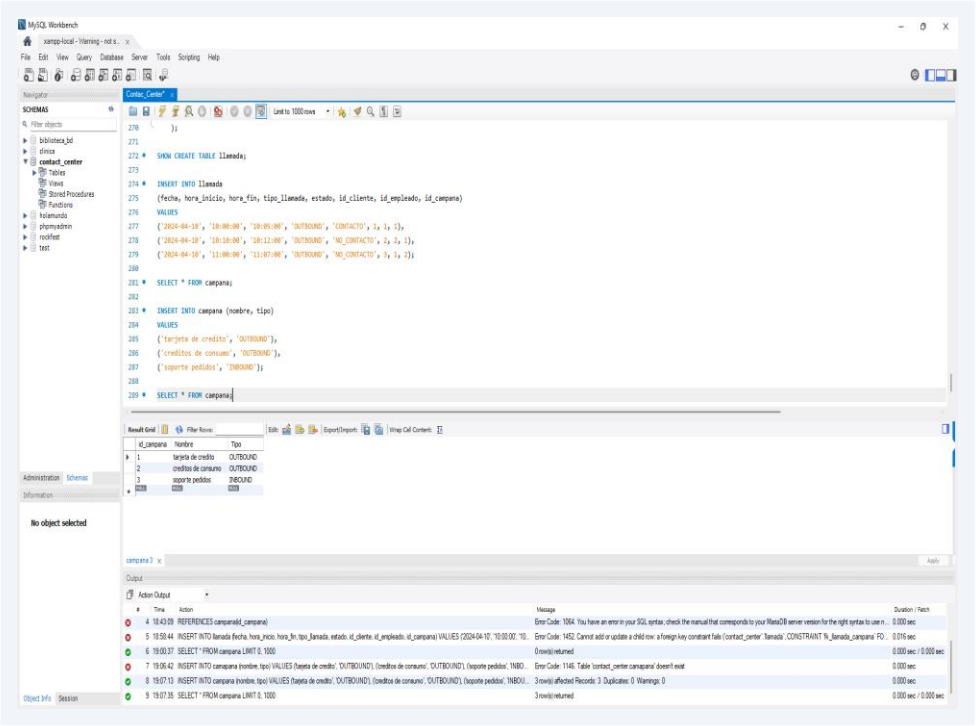
Se insertan datos coherentes: 3 clientes, 3 agentes (uno promovido a supervisor), 3 campañas (2 OUTBOUND de venta y 1 INBOUND de soporte) y varias llamadas con distintos resultados.

Por qué

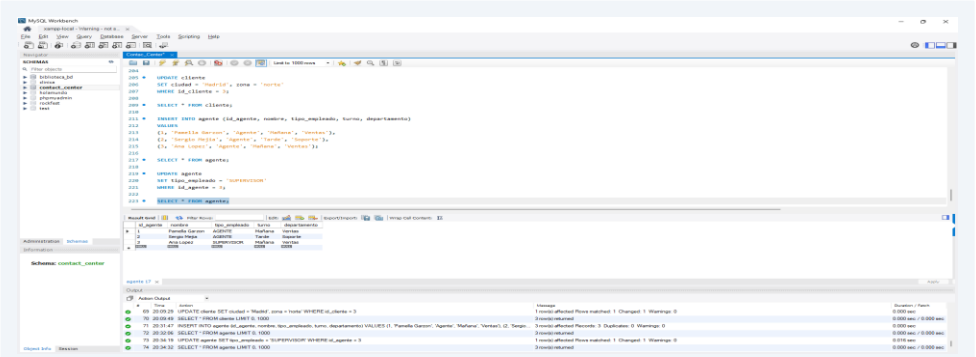
Insertar datos falsos te da una BD vacía de valor. Con datos parecidos a los del sector se pueden validar las consultas igual que un analista junior en su día a día.

Puntos clave

- Orden de inserción: primero entidades, después dependientes.
- ALTER TABLE para añadir telefono y email sin perder datos.
- TRIM y LENGTH para limpiar valores en ENUM.
- UPDATE controlado para promocionar a SUPERVISOR, sin rehacer tablas.



Inserciones y verificación con SELECT.



Consultas analíticas y KPIs

FASE
9

Qué hicimos

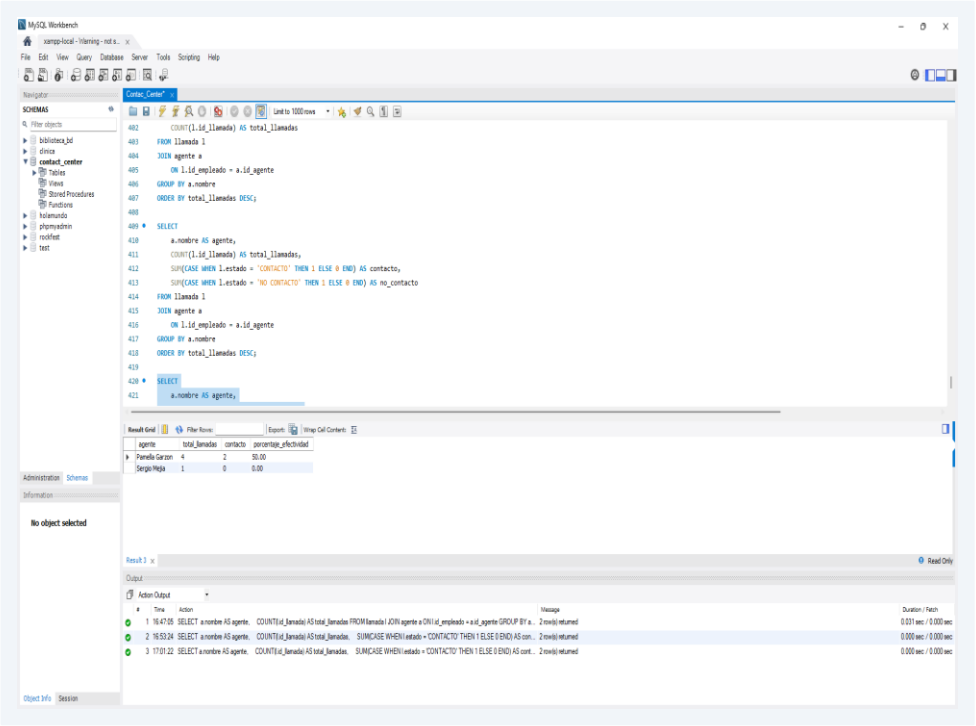
Se programan tres consultas clave: total de llamadas por agente, contactos vs no contactos y porcentaje de efectividad redondeado a dos decimales.

Por qué

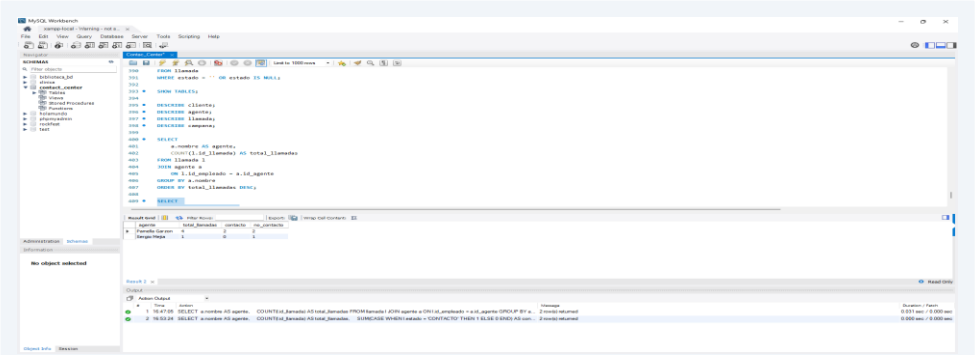
Un contact center vive de KPIs. Saber cuántas llamadas hace un agente es útil, pero saber su % de conversión es lo que marca la diferencia al evaluar o reasignar campañas.

Puntos clave

- JOIN entre llamada y agente por id_empleado / id_agente.
- COUNT + GROUP BY para el volumen.
- SUM(CASE WHEN ...) para separar CONTACTO y NO CONTACTO.
- ROUND((contactos / total) * 100, 2) para la efectividad.



Consulta de efectividad por agente.



Backup, usuarios y permisos

FASE
10

Qué hicimos

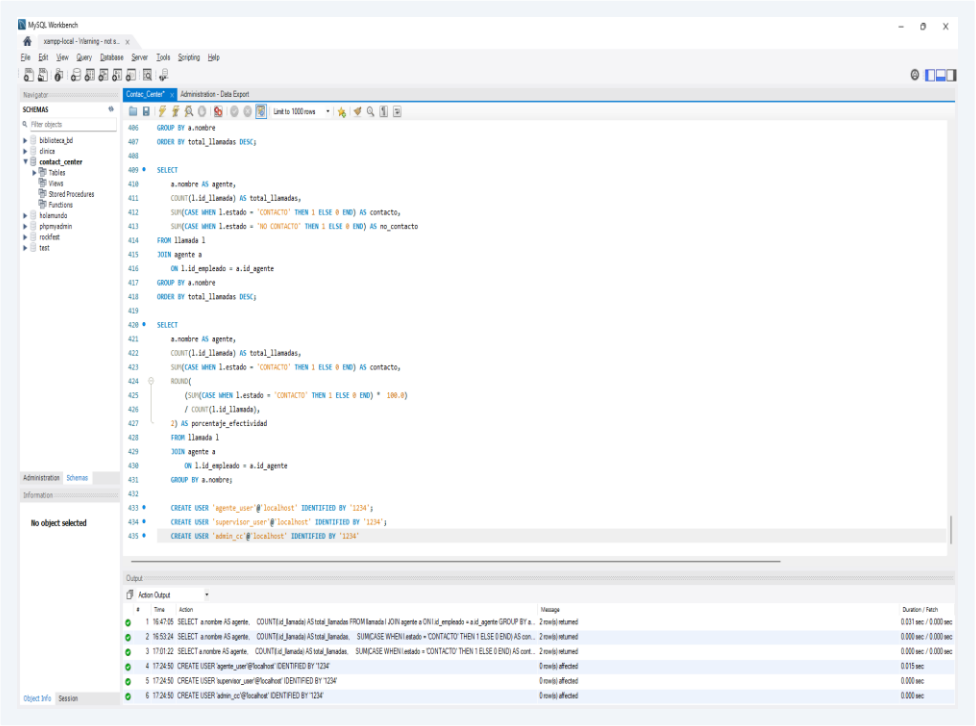
Se genera un backup completo con Data Export de Workbench y se crean tres usuarios con permisos distintos: agente_user, supervisor_user y admin_cc.

Por qué

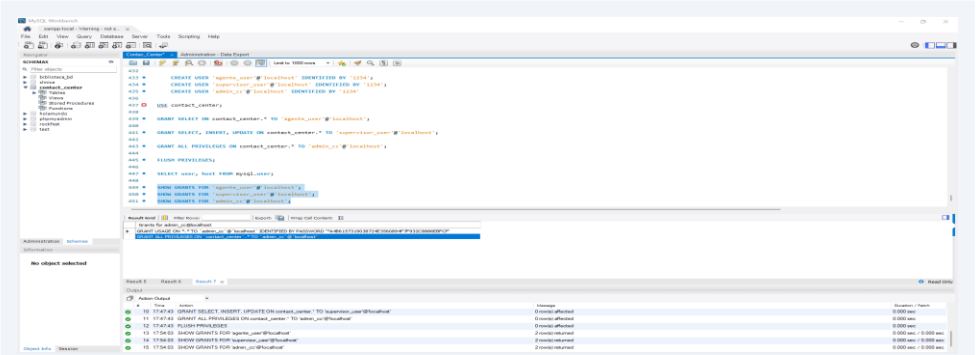
Una base de datos sin copia ni control de acceso es un problema esperando a pasar. Aquí se aplica el principio de mínimo privilegio: cada usuario solo puede hacer lo necesario para su rol.

Puntos clave

- Backup self-contained .sql (estructura + datos).
- agente_user: solo SELECT.
- supervisor_user: SELECT, INSERT, UPDATE.
- admin_cc: ALL PRIVILEGES. FLUSH PRIVILEGES para aplicar.



Creación de usuarios y GRANTs por rol.



Pruebas de funcionamiento

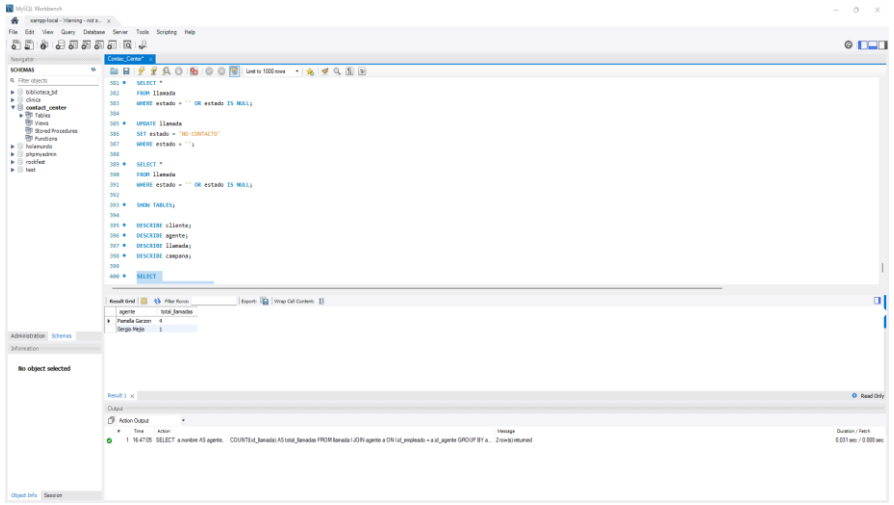
Tras cada fase se verificó que la BD se comportara como se esperaba. Las pruebas no se dejaron para el final: se hicieron a medida que se construía.

Tipos de prueba realizadas

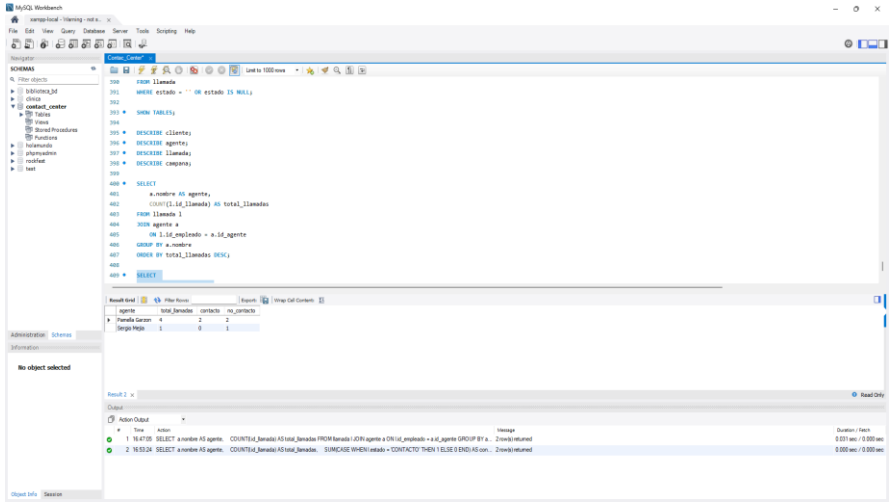
· SELECT * sobre cada tabla para confirmar que los datos coinciden con lo insertado.
· SHOW CREATE TABLE para revisar la estructura final, claves e índices.
· Intento de INSERT con FK inexistente para comprobar la integridad referencial.
· LENGTH() sobre ENUMs para detectar espacios sobrantes o valores fuera de lista.
· Consultas JOIN cruzando las tres claves foráneas de llamada.

Resultado

Todas las operaciones del negocio (insertar llamada, consultar KPIs, revisar permisos) funcionan de extremo a extremo.



Consulta de total de llamadas por agente.



Contactos vs no contactos.

Validación de resultados

¿Los datos dicen lo que deben decir?

Lo que mide el sistema

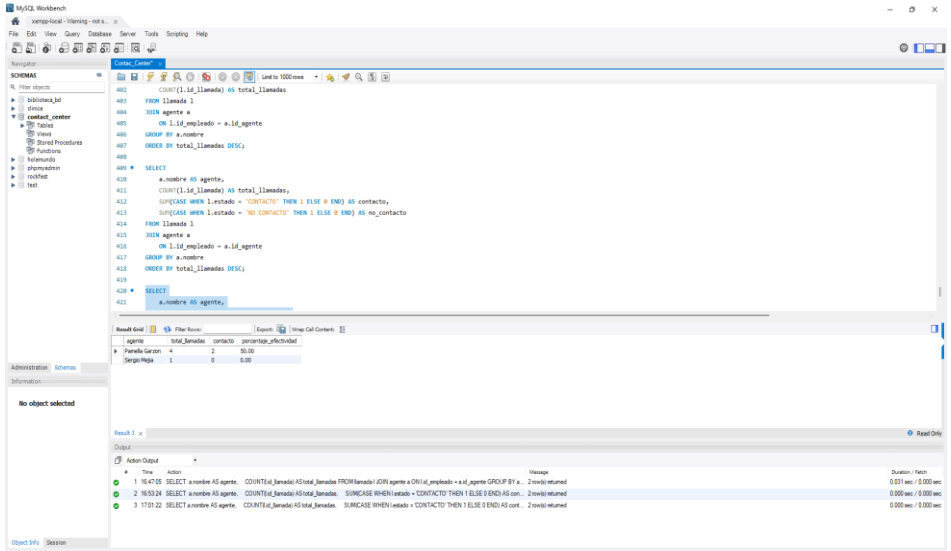
La consulta de efectividad devuelve un porcentaje por agente que coincide manualmente con los datos insertados. Al cambiar una llamada de NO CONTACTO a CONTACTO, el porcentaje sube como debe.

Lo que mide el control de acceso

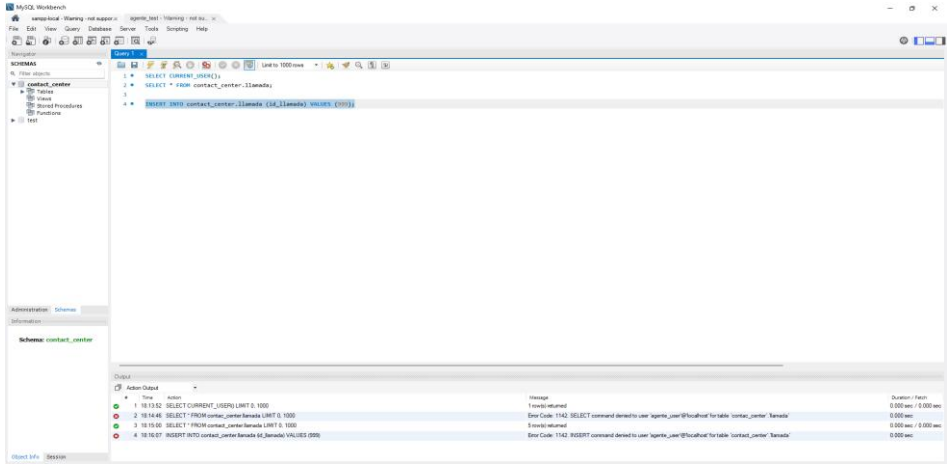
Al conectarse como agente_user, las sentencias INSERT fallan con error de permisos, mientras que SELECT funciona. Es exactamente lo que buscábamos.

Lo que se revisa con SHOW GRANTS

Cada usuario muestra únicamente los privilegios que le corresponden sobre contact_center.*



Efectividad por agente (%)



SHOW GRANTS — validación de permisos.

Problemas encontrados y soluciones

Lo que no salió a la primera

Problema: CREATE DATABASE con typo (contac_center).

Solución: Leer el mensaje de error real, corregir y volver a ejecutar.

Problema: Inserts en cliente fallando por telefono/email inexistentes.

Solución: ALTER TABLE … ADD COLUMN sin perder los datos ya insertados.

Problema: Inconsistencia mayúsculas/minúsculas en id_Campana / id_campana.

Solución: Normalización completa a snake_case con ALTER TABLE CHANGE.

Problema: Espacios en blanco detectados dentro de ENUM.

Solución: TRIM() y LENGTH() para detectar y corregir los registros.

Problema: Tabla con tilde (campaña) dio problemas.

Solución: DROP TABLE y recrear como campana, sin caracteres especiales.

Problema: Claves foráneas con nombres automáticos (llamada_ibfk_1).

Solución: DROP FOREIGN KEY + ADD CONSTRAINT con nombres propios.

Problema: SAFE UPDATES bloqueaba UPDATE masivos.

Solución: SET SQL_SAFE_UPDATES = 0 solo en el momento necesario.

Problema: Usuario agente intentando insertar.

Solución: Validado: el sistema lo rechaza. El modelo de permisos funciona.

Buenas prácticas aplicadas

Lo que hace que esto se parezca a un entorno real

1 Nomenclatura snake_case

Todas las tablas y campos en minúsculas con guion bajo.

2 Claves foráneas con nombre propio

fk_<tabla>_<referencia>, fáciles de leer y mantener.

3 ENUM para dominios cerrados

INBOUND/OUTBOUND y CONTACTO/NO CONTACTO para evitar datos sueltos.

4 Integridad referencial

Toda relación se respalda con FOREIGN KEY.

5 Índices explícitos

idx_llamada_* para acelerar las consultas por agente y campaña.

6 Mínimo privilegio

Cada usuario solo tiene los permisos que necesita para su rol.

7 Backup reproducible

Archivo .sql self-contained que restaura todo en cualquier servidor.

8 Documentación continua

Cada decisión queda anotada en el script y en el repositorio.

Conclusiones

El proyecto deja claro algo importante: diseñar una base de datos no es solo crear tablas. Es entender el negocio, pensar cómo se conecta la información y dejar el entorno preparado para que otras personas puedan usarlo con seguridad.

Trabajar sobre un Contact Center no fue casualidad. Elegí este dominio porque llevo tiempo rodeado de este tipo de operaciones de forma profesional, y eso me ha permitido tomar decisiones técnicas con criterio: por qué los estados son un ENUM, por qué la efectividad se mide con CASE WHEN, por qué separamos supervisores de agentes y por qué un agente no debería poder borrar una llamada.

El resultado es una base de datos funcional, normalizada y respaldada, con consultas que aportan valor real y un control de accesos que respeta el principio de mínimo privilegio. Lo que en clase se ve teórico, aquí se ve aplicado a un caso que me importa.

PROYECTO INTERMODULAR

Gracias

Gestion de Bases de Datos – ASIR 1º

Administración de Sistemas Informáticos en Red

Sergio A. Tellez Tellez